Kwalifikacja INF.04

Zestaw zadań ćwiczeniowych

Aplikacje internetowe (React)

---

12 zadań React • wersja dla ucznia

Klasa 5 technikum – przygotowanie do egzaminu zawodowego

Rok szkolny 2026/2027

Spis treści

1. Wymagania wspólne dla wszystkich zadań 3
1.1. Aplikacje internetowe (React) 3
1.2. Dokumentacja 3
2. Przegląd zadań 4
3. Zadania – aplikacje internetowe (React) 5
Zadanie R1. Zapis na kurs 5
Zadanie R2. Galeria zdjęć z filtrem i licznikiem pobrań 6
Zadanie R3. Moja lista filmów 7
Zadanie R4. Lista zadań (To-Do) 8
Zadanie R5. Kalkulator zapotrzebowania kalorycznego 9
Zadanie R6. Koszyk sklepu komputerowego 10
Zadanie R7. Tabela uczniów – wyszukiwanie i sortowanie 11
Zadanie R8. Quiz jednokrotnego wyboru 12
Zadanie R9. Tablica ogłoszeń z polubieniami 13
Zadanie R10. Rejestracja z walidacją na żywo 14
Zadanie R11. Przelicznik jednostek 15
Zadanie R12. Dziennik ocen – średnia ważona 16

Aby odświeżyć spis treści w programie Word: kliknij prawym przyciskiem myszy na spis i wybierz „Aktualizuj pole”.

Jak pracować z zestawem

Zestaw zawiera 12 zadań przygotowujących do części egzaminu INF.04 dotyczącej aplikacji internetowych. Zadania są napisane tak jak w arkuszu egzaminacyjnym: czytaj treść dokładnie i realizuj każdy wymóg – na egzaminie oceniany jest każdy z nich osobno.

- ★☆☆ zadanie podstawowe, ★★☆ średnie, ★★★ rozszerzone.
- W miejsce <numer zdającego> wpisz swój numer z dziennika (lub identyfikator podany przez nauczyciela).
- Wymagania wspólne z punktu 1 obowiązują w każdym zadaniu, nawet jeśli treść zadania ich nie powtarza.
- Jeśli w zadaniu podano dane testowe, sprawdź na nich swój program – wynik musi się zgadzać.
- Na egzaminie na jedną aplikację masz w praktyce około 45–60 minut. Mierz czas swojej pracy.

1. Wymagania wspólne dla wszystkich zadań

Poniższe wymagania obowiązują w każdym zadaniu, nawet jeśli treść zadania ich nie powtarza – tak jak w arkuszu egzaminacyjnym.

1.1. Aplikacje internetowe (React)

- Aplikacja utworzona w bibliotece React, np. poleceniem npm create vite@latest (szablon React), uruchamiana poleceniem npm run dev.
- Do stylowania używana jest biblioteka Bootstrap (npm install bootstrap oraz import "bootstrap/dist/css/bootstrap.min.css").
- Komponenty funkcyjne, stan przechowywany z użyciem hooka useState; dane podane w treści umieszczone w kodzie komponentu.
- Wygląd uzyskany klasami Bootstrap; własne style CSS tylko tam, gdzie treść zadania tego wymaga.
- Elementy list generowane metodą map muszą mieć unikalny atrybut key.

1.2. Dokumentacja

Wskazaną w zadaniu funkcję należy poprzedzić komentarzem według wzoru:

```
/*************************************************
 nazwa funkcji:        <tu wstaw nazwę>
 opis funkcji:         <tu wstaw opis działania>
 parametry:            <tu wstaw opis parametrów lub „brak”>
 zwracany typ i opis:  <tu wstaw typ i opis lub „brak”>
 autor:                <numer zdającego>
*************************************************/
```

- Zrzuty ekranu wykonuje się w formacie PNG i nazywa zgodnie z treścią zadania (web1.png, web2.png itd.). Zrzut obejmuje całe okno przeglądarki; jeśli zadanie wymaga pokazania konsoli, powinny być widoczne jednocześnie strona i narzędzia deweloperskie.
- Projekt i zrzuty ekranu zapisuje się w folderze nazwanym numerem zdającego, w podfolderze web.
- Zalecane ćwiczenie dodatkowe: krótki opis testów – jakie dane wprowadzono i jakiego wyniku oczekiwano.

2. Przegląd zadań

Nr	Zadanie	Poziom	Kluczowe umiejętności
R1	Zapis na kurs	★☆☆	renderowanie listy (map), useState, formularz, console.log, preventDefault
R2	Galeria zdjęć z filtrem i licznikiem pobrań	★★☆	filtrowanie tablicy, niemutowalna aktualizacja stanu, checkbox, siatka Bootstrap
R3	Moja lista filmów	★☆☆	select, walidacja z klasami Bootstrap (is-invalid), renderowanie tabeli
R4	Lista zadań (To-Do)	★★☆	dodawanie/usuwanie/edycja elementów stanu, filtrowanie, obsługa klawisza Enter
R5	Kalkulator zapotrzebowania kalorycznego	★☆☆	radio, select z wartościami liczbowymi, obliczenia, warunkowe klasy CSS
R6	Koszyk sklepu komputerowego	★★☆	stan złożony (tablica obiektów), reduce, formatowanie kwot, kod rabatowy
R7	Tabela uczniów – wyszukiwanie i sortowanie	★★☆	filter, sort na kopii tablicy, stan kierunku sortowania, klasy warunkowe wierszy
R8	Quiz jednokrotnego wyboru	★★☆	renderowanie warunkowe, stan wieloetapowy, progress bar
R9	Tablica ogłoszeń z polubieniami	★★☆	textarea z licznikiem znaków, blokowanie przycisku, sortowanie wg różnych kryteriów
R10	Rejestracja z walidacją na żywo	★★★	wyrażenia regularne, walidacja przy każdej zmianie, obiekt stanu formularza, wskaźnik siły hasła
R11	Przelicznik jednostek	★☆☆	stan zależny (reset jednostek), obliczenia w trakcie renderowania, obiekty współczynników
R12	Dziennik ocen – średnia ważona	★★☆	zakładki (nav-tabs), filtrowanie po kategorii, średnia ważona, progi ocen

3. Zadania – aplikacje internetowe (React)

Zadania R1–R12. Przed rozpoczęciem pracy zapoznaj się z wymaganiami wspólnymi (punkt 1).

Zadanie R1. Zapis na kurs

Poziom: ★☆☆ **Umiejętności:** renderowanie listy (map), useState, formularz, console.log, preventDefault

Przygotuj aplikację internetową umożliwiającą zapis na kurs programowania.

Dane

Umieść w kodzie aplikacji poniższe dane:

```
const kursy = ["Programowanie w C#", "Angular dla początkujących",
  "Kurs Django", "Tworzenie aplikacji w React", "Bazy danych SQL"];
```

Wygląd strony

- Nagłówek h2 „Liczba kursów: N”, gdzie N to długość tablicy (wyliczane, nie wpisane na sztywno).
- Lista numerowana (ol) z nazwami kursów generowana metodą map.
- Formularz w stylu Bootstrap: pola „Imię i nazwisko” (text) oraz „Numer kursu” (number), klasy form-control.
- Przycisk „Zapisz do kursu” z klasą btn btn-primary.

Działanie strony

- Wartości pól przechowywane w stanie komponentu (useState).
- Po kliknięciu przycisku strona nie jest przeładowywana (preventDefault).
- W konsoli wypisywane jest imię i nazwisko.
- Jeśli numer kursu mieści się w zakresie 1–N, w konsoli wypisywana jest nazwa kursu; w przeciwnym razie „Nieprawidłowy numer kursu”.

Dokumentacja

Wykonaj zrzuty ekranu:

- web1.png – aplikacja z wypełnionym formularzem
- web2.png – konsola z poprawnym zapisem
- web3.png – konsola z błędnym numerem

Opatrz funkcję obsługi zdarzenia komentarzem zgodnym ze wzorem z punktu 1.2.

Zadanie R2. Galeria zdjęć z filtrem i licznikiem pobrań

Poziom: ★★☆ **Umiejętności:** filtrowanie tablicy, niemutowalna aktualizacja stanu, checkbox, siatka Bootstrap

Przygotuj galerię zdjęć, w której użytkownik filtruje zdjęcia według kategorii i może je pobierać, zwiększając licznik pobrań.

Dane

Umieść w kodzie aplikacji poniższe dane:

```
const zdjecia = [
  { id: 0, alt: "Tatry", filename: "obraz1.jpg", category: 1, downloads: 35 },
  { id: 1, alt: "Bieszczady", filename: "obraz2.jpg", category: 1, downloads: 43 },
  { id: 2, alt: "Plaża w Łebie", filename: "obraz3.jpg", category: 2, downloads: 12 },
  { id: 3, alt: "Klif w Orzechowie", filename: "obraz4.jpg", category: 2, downloads: 27 },
  { id: 4, alt: "Rynek w Krakowie", filename: "obraz5.jpg", category: 3, downloads: 51 },
  { id: 5, alt: "Spodek w Katowicach", filename: "obraz6.jpg", category: 3, downloads: 19 }
]; // kategorie: 1 – Góry, 2 – Morze, 3 – Miasto
```

Wygląd strony

- Nagłówek h1 „Kategorie zdjęć”.
- Trzy przełączniki (form-check form-switch): „Góry”, „Morze”, „Miasto” – domyślnie wszystkie zaznaczone.
- Zdjęcia w siatce Bootstrap (row, col-md-4), każde w karcie (card) zawierającej obraz z atrybutem alt, nagłówek h4 „Pobrań: N” i przycisk „Pobierz” (btn btn-success).
- Pliki obrazów (obraz1.jpg – obraz6.jpg) umieść w folderze public/img.

Działanie strony

- Wyświetlane są tylko zdjęcia z zaznaczonych kategorii; zmiana przełącznika natychmiast odświeża widok.
- Kliknięcie „Pobierz” zwiększa o 1 licznik pobrań wyłącznie tego zdjęcia.
- Aktualizacja stanu nie modyfikuje tablicy bezpośrednio (map + kopia obiektu).
- Gdy żadna kategoria nie jest zaznaczona, wyświetlany jest komunikat „Wybierz co najmniej jedną kategorię”.

Dokumentacja

Wykonaj zrzuty ekranu:

- web1.png – wszystkie kategorie
- web2.png – odznaczona kategoria „Morze”, zwiększony licznik jednego zdjęcia

Opatrz funkcję pobierania komentarzem zgodnym ze wzorem z punktu 1.2.

Zadanie R3. Moja lista filmów

Poziom: ★☆☆ **Umiejętności:** select, walidacja z klasami Bootstrap (is-invalid), renderowanie tabeli

Przygotuj aplikację do tworzenia listy filmów do obejrzenia.

Wygląd strony

- Formularz: pole „Tytuł filmu” (form-control) oraz lista „Rodzaj” (form-select) z opcjami: 1 – Komedia, 2 – Obyczajowy, 3 – Sensacyjny, 4 – Horror.
- Przycisk „Dodaj” (btn btn-primary).
- Pod formularzem nagłówek h3 „Moja lista filmów” i tabela (table table-striped) z kolumnami: Lp., Tytuł, Rodzaj.
- Gdy lista jest pusta, zamiast tabeli wyświetlany jest akapit „Brak filmów na liście”.

Działanie strony

- Po kliknięciu „Dodaj” w konsoli wypisywany jest tekst „tytuł: <tytuł>; rodzaj: <numer>”.
- Film trafia do tabeli, a w kolumnie „Rodzaj” wyświetlana jest nazwa rodzaju (nie numer).
- Pusty tytuł: pole otrzymuje klasę is-invalid, pod nim widoczny jest komunikat invalid-feedback „Podaj tytuł filmu”, film nie jest dodawany.
- Po dodaniu formularz jest czyszczony.

Dokumentacja

Wykonaj zrzuty ekranu:

- web1.png – tabela z co najmniej trzema filmami
- web2.png – błąd pustego tytułu
- web3.png – konsola

Opatrz funkcję dodającą komentarzem zgodnym ze wzorem z punktu 1.2.

Zadanie R4. Lista zadań (To-Do)

Poziom: ★★☆ **Umiejętności:** dodawanie/usuwanie/edycja elementów stanu, filtrowanie, obsługa klawisza Enter

Przygotuj aplikację do zarządzania listą zadań.

Wygląd strony

- Pole tekstowe i przycisk „Dodaj” w jednym wierszu (input-group).
- Grupa przycisków (btn-group): „Wszystkie”, „Aktywne”, „Wykonane” – aktywny filtr wyróżniony klasą active.
- Lista zadań (list-group); każde zadanie: pole wyboru, treść, przycisk „Usuń” (btn btn-danger btn-sm).
- Pod listą tekst „Pozostało do zrobienia: X z Y”.

Działanie strony

- Zadanie dodawane przyciskiem lub klawiszem Enter; puste zadania nie są dodawane.
- Każde zadanie jest obiektem { id, tresc, wykonane }.
- Zaznaczenie pola wyboru oznacza zadanie jako wykonane – treść przekreślona i wyszarzona (text-decoration-line-through, text-muted).
- Filtry pokazują odpowiednio wszystkie, niewykonane lub wykonane zadania.
- Licznik aktualizowany na bieżąco.

Dokumentacja

Wykonaj zrzuty ekranu:

- web1.png – lista z zadaniami wykonanymi i niewykonanymi
- web2.png – włączony filtr „Aktywne”

Opatrz funkcję zmiany stanu zadania komentarzem zgodnym ze wzorem z punktu 1.2.

Zadanie R5. Kalkulator zapotrzebowania kalorycznego

Poziom: ★☆☆ **Umiejętności:** radio, select z wartościami liczbowymi, obliczenia, warunkowe klasy CSS

Przygotuj aplikację obliczającą dzienne zapotrzebowanie kaloryczne wzorem Mifflina-St Jeora.

Wygląd strony

- Karta Bootstrap (card) o szerokości maks. 500 px, wyśrodkowana, nagłówek „Kalkulator kalorii".
- Przyciski opcji „Kobieta" / „Mężczyzna".
- Pola liczbowe: „Wiek [lata]", „Masa [kg]", „Wzrost [cm]".
- Lista „Aktywność": Siedzący tryb życia (1,2), Lekka aktywność (1,375), Umiarkowana (1,55), Wysoka (1,725).
- Przycisk „Oblicz" i miejsce na wynik w komponencie alert.

Działanie strony

- Podstawowa przemiana materii: kobieta 10·masa + 6,25·wzrost − 5·wiek − 161; mężczyzna 10·masa + 6,25·wzrost − 5·wiek + 5.
- Wynik = przemiana podstawowa × współczynnik aktywności, zaokrąglony do liczby całkowitej.
- Wynik w formie „Twoje zapotrzebowanie: 2250 kcal/dzień" w komponencie alert-success.
- Walidacja: wiek 15–100, masa 30–250, wzrost 120–230; przy błędzie alert-danger „Sprawdź wprowadzone dane".
- Test kontrolny: mężczyzna, 18 lat, 70 kg, 180 cm, umiarkowana aktywność → 2697 kcal.

Dokumentacja

Wykonaj zrzuty ekranu:

- web1.png – wynik dla danych testowych
- web2.png – komunikat o błędnych danych

Opatrz funkcję obliczającą komentarzem zgodnym ze wzorem z punktu 1.2.

Zadanie R6. Koszyk sklepu komputerowego

Poziom: ★★☆ **Umiejętności:** stan złożony (tablica obiektów), reduce, formatowanie kwot, kod rabatowy

Przygotuj stronę sklepu z listą produktów i koszykiem.

Dane

Umieść w kodzie aplikacji poniższe dane:

```
const produkty = [
  { id: 1, nazwa: "Klawiatura mechaniczna", cena: 249.99 },
  { id: 2, nazwa: "Mysz bezprzewodowa", cena: 89.50 },
  { id: 3, nazwa: "Monitor 24 cale", cena: 699.00 },
  { id: 4, nazwa: "Słuchawki", cena: 159.90 },
  { id: 5, nazwa: "Pendrive 64 GB", cena: 39.99 }
];
```

Wygląd strony

- Układ dwukolumnowy (row, col-md-7 i col-md-5).
- Lewa kolumna: produkty w kartach z nazwą, ceną i przyciskiem „Dodaj do koszyka”.
- Prawa kolumna „Koszyk”: tabela z kolumnami Produkt, Ilość (przyciski „−” i „+” wokół liczby), Wartość.
- Pod tabelą pole „Kod rabatowy” z przyciskiem „Zastosuj” oraz wiersz „Do zapłaty: X zł” (pogrubiony).

Działanie strony

- Dodanie produktu, który już jest w koszyku, zwiększa jego ilość.
- Przycisk „−” przy ilości 1 usuwa produkt z koszyka.
- Suma obliczana metodą reduce; kwoty wyświetlane z dwoma miejscami po przecinku.
- Kod „INF04” obniża sumę o 10% i wyświetla informację „Rabat 10% naliczony”; inny kod – „Nieprawidłowy kod”.
- Pusty koszyk – komunikat „Koszyk jest pusty” zamiast tabeli.

Dokumentacja

Wykonaj zrzuty ekranu:

- web1.png – koszyk z trzema produktami, jeden w ilości większej niż 1
- web2.png – naliczony rabat

Opatrz funkcję dodającą do koszyka komentarzem zgodnym ze wzorem z punktu 1.2.

Zadanie R7. Tabela uczniów – wyszukiwanie i sortowanie

Poziom: ★★☆ **Umiejętności:** filter, sort na kopii tablicy, stan kierunku sortowania, klasy warunkowe wierszy

Przygotuj stronę prezentującą listę uczniów z możliwością wyszukiwania, filtrowania i sortowania.

Dane

Umieść w kodzie aplikacji poniższe dane:

```
const uczniowie = [
  { id: 1, imie: "Anna", nazwisko: "Nowak", klasa: "5TI", srednia: 4.85 },
  { id: 2, imie: "Piotr", nazwisko: "Kowalski", klasa: "5TP", srednia: 3.42 },
  { id: 3, imie: "Julia", nazwisko: "Wiśniewska", klasa: "5TI", srednia: 5.10 },
  { id: 4, imie: "Kacper", nazwisko: "Wójcik", klasa: "5TP", srednia: 2.95 },
  { id: 5, imie: "Zofia", nazwisko: "Kamińska", klasa: "5TI", srednia: 4.20 },
  { id: 6, imie: "Jakub", nazwisko: "Lewandowski", klasa: "5TP", srednia: 4.76 },
  { id: 7, imie: "Maja", nazwisko: "Zielińska", klasa: "5TI", srednia: 3.88 },
  { id: 8, imie: "Filip", nazwisko: "Szymański", klasa: "5TP", srednia: 3.05 }
];
```

Wygląd strony

- Pole „Szukaj po nazwisku” oraz lista „Klasa” (Wszystkie, 5TI, 5TP) w jednym wierszu.
- Tabela (table table-hover) z kolumnami: Lp., Imię, Nazwisko, Klasa, Średnia.
- Nagłówki „Nazwisko” i „Średnia” są klikalne i pokazują strzałkę ▲lub ▼przy aktywnym sortowaniu.
- Pod tabelą tekst „Wyświetlono X z Y uczniów”.

Działanie strony

- Wyszukiwanie działa na bieżąco podczas pisania i nie rozróżnia wielkości liter.
- Filtr klasy łączy się z wyszukiwaniem.
- Kliknięcie nagłówka sortuje rosnąco, kolejne kliknięcie – malejąco. Sortowanie nazwisk uwzględnia polskie znaki (localeCompare).
- Wiersze uczniów ze średnią co najmniej 4,75 mają klasę table-success.
- Średnia wyświetlana z dwoma miejscami po przecinku.

Dokumentacja

Wykonaj zrzuty ekranu:

- web1.png – tabela posortowana malejąco według średniej
- web2.png – wyszukiwanie i filtr klasy jednocześnie

Opatrz funkcję sortującą komentarzem zgodnym ze wzorem z punktu 1.2.

Zadanie R8. Quiz jednokrotnego wyboru

Poziom: ★★☆ **Umiejętności:** renderowanie warunkowe, stan wieloetapowy, progress bar

Przygotuj aplikację quizu, w której pytania wyświetlane są pojedynczo.

Dane

Umieść w kodzie aplikacji poniższe dane:

```
const pytania = [
  { tresc: "Który znacznik HTML tworzy hiperłącze?", odp: ["<link>", "<a>", "<href>", "<url>"],
poprawna: 1 },
  { tresc: "Który operator w JS porównuje wartość i typ?", odp: ["==", "=", "===", "!="], poprawna:
2 },
  { tresc: "Jak nazywa się klasa bazowa wszystkich typów w C#?", odp: ["Object", "Base", "Root",
"Type"], poprawna: 0 },
  { tresc: "Które polecenie Git zapisuje zmiany w repozytorium lokalnym?", odp: ["push", "pull",
"clone", "commit"], poprawna: 3 },
  { tresc: "Który hook React wykonuje efekt uboczny?", odp: ["useState", "useEffect", "useMemo",
"useId"], poprawna: 1 }
];
```

Wygląd strony

- Karta z nagłówkiem „Pytanie N z 5” i paskiem postępu (progress-bar) odpowiadającym numerowi pytania.
- Treść pytania w h4, pod nią cztery przyciski odpowiedzi na całą szerokość (btn btn-outline-primary w-100 mb-2).
- Przycisk „Dalej” widoczny dopiero po wybraniu odpowiedzi.
- Ekran końcowy: wynik punktowy i procentowy oraz przycisk „Zagraj ponownie”.

Działanie strony

- Po wybraniu odpowiedzi poprawna otrzymuje klasę btn-success, a błędnie wybrana – btn-danger; pozostałe przyciski są zablokowane.
- „Dalej” przechodzi do kolejnego pytania, po ostatnim – do ekranu wyniku.
- Wynik: „Twój wynik: 4/5 (80%)”; przy wyniku co najmniej 80% dodatkowy tekst „Gratulacje!”.
- „Zagraj ponownie” resetuje quiz do pierwszego pytania.

Dokumentacja

Wykonaj zrzuty ekranu:

- web1.png – pytanie po udzieleniu błędnej odpowiedzi
- web2.png – ekran wyniku

Opatrz funkcję obsługi odpowiedzi komentarzem zgodnym ze wzorem z punktu 1.2.

Zadanie R9. Tablica ogłoszeń z polubieniami

Poziom: ★★☆ **Umiejętności:** textarea z licznikiem znaków, blokowanie przycisku, sortowanie wg różnych kryteriów

Przygotuj tablicę ogłoszeń szkolnych, na której można dodawać wpisy i je polubić.

Dane

Umieść w kodzie aplikacji poniższe dane:

```
const startowePosty = [
  { id: 1, autor: "Samorząd", tresc: "Dzień Sportu w piątek!", polubienia: 12, data: "2026-09-10" },
  { id: 2, autor: "Biblioteka", tresc: "Nowe książki o programowaniu.", polubienia: 4, data: "2026-
09-14" },
  { id: 3, autor: "Klasa 5TI", tresc: "Szukamy chętnych do hackathonu.", polubienia: 9, data: "2026-
09-18" }
];
```

Wygląd strony

- Formularz: pole „Autor”, pole wielowierszowe „Treść” (textarea, 3 wiersze), pod nim licznik „N/200”, przycisk „Opublikuj”.
- Lista „Sortuj”: Najnowsze, Najpopularniejsze.
- Wpisy w kartach: autor (pogrubiony) i data (text-muted, mała czcionka), treść, przyciski „ Lubię (N)” (btn-outline-primary) i „Usuń” (btn-outline-danger).

Działanie strony

- Licznik znaków aktualizowany podczas pisania; powyżej 180 znaków ma kolor czerwony (text-danger).
- „Opublikuj” jest zablokowany, gdy autor lub treść są puste albo treść przekracza 200 znaków.
- Nowy wpis otrzymuje bieżącą datę i 0 polubień.
- „Lubię” zwiększa licznik polubień danego wpisu.
- Sortowanie według daty (najnowsze pierwsze) lub liczby polubień (malejąco).
- „Usuń” pyta o potwierdzenie (window.confirm).

Dokumentacja

Wykonaj zrzuty ekranu:

- web1.png – tablica posortowana wg popularności
- web2.png – formularz z licznikiem powyżej 180 znaków

Opatrz funkcję publikującą komentarzem zgodnym ze wzorem z punktu 1.2.

Zadanie R10. Rejestracja z walidacją na żywo

Poziom: ★★★ **Umiejętności:** wyrażenia regularne, walidacja przy każdej zmianie, obiekt stanu formularza, wskaźnik siły hasła

Przygotuj formularz rejestracji użytkownika, w którym każde pole jest sprawdzane podczas wpisywania.

Wygląd strony

- Formularz w karcie o szerokości maks. 450 px: „Login”, „E-mail”, „Hasło”, „Powtórz hasło”, pole wyboru „Akceptuję regulamin”.
- Pod polem hasła pasek postępu „Siła hasła” z opisem: słabe (czerwony), średnie (żółty), silne (zielony).
- Przycisk „Zarejestruj” (btn btn-primary w-100).

Działanie strony

- Login: co najmniej 4 znaki, tylko litery i cyfry.
- E-mail: zgodny z wyrażeniem regularnym /^[^\s@]+@[^\s@]+\.[^\s@]+$/.
- Hasło: co najmniej 8 znaków, co najmniej jedna wielka litera i jedna cyfra; powtórzenie identyczne z hasłem.
- Każde pole po pierwszej modyfikacji otrzymuje klasę is-valid lub is-invalid; komunikaty błędów w invalid-feedback.
- Siła hasła: liczba spełnionych kryteriów spośród 4 (długość ≥ 8, wielka litera, cyfra, znak specjalny); 0–1 słabe, 2–3 średnie, 4 silne.
- „Zarejestruj” aktywny tylko, gdy wszystkie pola są poprawne i regulamin zaakceptowany. Po kliknięciu: alert-success „Konto <login> zostało utworzone” oraz w konsoli obiekt { login, email } (bez hasła).

Dokumentacja

Wykonaj zrzuty ekranu:

- web1.png – formularz z błędami
- web2.png – poprawnie wypełniony formularz z silnym hasłem
- web3.png – komunikat po rejestracji i konsola

Opatrz funkcję walidującą komentarzem zgodnym ze wzorem z punktu 1.2.

Zadanie R11. Przelicznik jednostek

Poziom: ★☆☆ **Umiejętności:** stan zależny (reset jednostek), obliczenia w trakcie renderowania, obiekty współczynników

Przygotuj aplikację przeliczającą jednostki długości, masy i temperatury bez użycia przycisku „Oblicz”.

Wygląd strony

- Lista „Kategoria”: Długość, Masa, Temperatura.
- Pole „Wartość” (number), listy „Z jednostki” i „Na jednostkę” oraz przycisk „⇄.
- Wynik w nagłówku h3, np. „1 km = 1000 m”.

Działanie strony

- Długość: m, km, cm, mm, cal (0,0254 m), stopa (0,3048 m). Masa: kg, g, t, funt (0,45359237 kg).
- Współczynniki przechowywane w obiektach względem jednostki bazowej (m, kg).
- Temperatura: °C, °F, K – przeliczana wzorami przez stopnie Celsjusza.
- Wynik obliczany na bieżąco przy każdej zmianie, zaokrąglony do 4 miejsc po przecinku (bez zbędnych zer).
- Zmiana kategorii ustawia domyślne jednostki tej kategorii.
- „⇄ zamienia jednostki miejscami. Test: 100 °C = 212 °F; 1 funt = 0,4536 kg.

Dokumentacja

Wykonaj zrzuty ekranu:

- web1.png – przeliczenie długości
- web2.png – przeliczenie temperatury

Opatrz funkcję przeliczającą komentarzem zgodnym ze wzorem z punktu 1.2.

Zadanie R12. Dziennik ocen – średnia ważona

Poziom: ★★☆ **Umiejętności:** zakładki (nav-tabs), filtrowanie po kategorii, średnia ważona, progi ocen

Przygotuj aplikację pozwalającą uczniowi kontrolować oceny i średnią ważoną z przedmiotów zawodowych.

Wygląd strony

- Zakładki (nav nav-tabs): „Aplikacje desktopowe”, „Aplikacje webowe”, „Bazy danych”.
- Formularz: lista „Ocena” (1–6), lista „Waga” (1–3), pole „Opis” (np. „Sprawdzian”), przycisk „Dodaj ocenę”.
- Tabela ocen wybranego przedmiotu: Ocena, Waga, Opis, przycisk „Usuń”.
- Pod tabelą „Średnia ważona: X” w odznace (badge) oraz „Przewidywana ocena roczna: Y”.

Działanie strony

- Ocena dodawana jest do przedmiotu z aktywnej zakładki; opis nie może być pusty.
- Średnia ważona = Σ(ocena × waga) / Σ wag, z dwoma miejscami po przecinku; bez ocen wyświetlane jest „–”.
- Kolor odznaki: poniżej 1,75 – bg-danger; od 4,75 – bg-success; w pozostałych przypadkach – bg-secondary.
- Przewidywana ocena: od 1,75 – 2; od 2,75 – 3; od 3,75 – 4; od 4,75 – 5; od 5,50 – 6; poniżej 1,75 – 1.
- Test: oceny 5 (waga 3), 4 (waga 2), 6 (waga 1) → średnia 4,83, ocena przewidywana 5.

Dokumentacja

Wykonaj zrzuty ekranu:

- web1.png – zakładka z co najmniej czterema ocenami
- web2.png – inna zakładka z odmiennym kolorem odznaki

Opatrz funkcję liczącą średnią komentarzem zgodnym ze wzorem z punktu 1.2.